ORE Studio User Manual

Marco Craveiro

Version 0.0.18

Contents

1 Introduction **2**

1.1 About ORE Studio . . . 2
1.2 Audience . . . 2
1.3 Functional Areas . . . 3
1.4 What ORE Studio Is Not . . . 3
1.5 How This Manual Is Organised . . . 3
1.6 Early-Stage Software Notice . . . 4
1.7 A Note on This Document . . . 4

2 Connecting to ORE Studio **5**

2.1 Getting started . . . 5
2.2 The main window before login . . . 5
2.3 The Connection Manager . . . 6
2.3.1 Adding a new connection . . . 7
2.3.2 Editing a connection . . . 7
2.3.3 Deleting a connection . . . 8
2.3.4 Organising connections into folders . . . 8
2.3.5 Protecting your credentials with a master password . . . 8
2.3.6 Environments . . . 10
2.3.7 Tags . . . 12
2.3.8 Where your connections are stored . . . 12
2.3.9 Backing up and restoring your connections . . . 13
2.4 Connecting and logging in . . . 13
2.4.1 Selecting the active connection . . . 13
2.4.2 The login screen . . . 14
2.4.3 First login and the default account . . . 15
2.5 Connection status and reconnection . . . 15
2.6 The status bar . . . 15
2.7 Starting ORE Studio from the command line . . . 16
2.7.1 Telling windows apart (instance colour and name) . . . 17
2.7.2 Configuration directory . . . 17
2.7.3 Selecting a connection at startup . . . 17

2.7.4 Logging and diagnostics . . . 17
2.8 Finding the application version . . . 18
2.9 Running in the background and the system tray . . . 18
2.10 Logging out . . . 18

3 Initial Setup **21**
3.1 Understanding the multi-tenant, multi-party model . . . 21
3.1.1 Tenants . . . 21
3.1.2 Parties . . . 22
3.1.3 The bootstrapping sequence . . . 22
3.2 The System Provisioner wizard . . . 23
3.2.1 Welcome page . . . 23
3.2.2 Administrator account . . . 23
3.2.3 Review and confirm . . . 24
3.2.4 Completion . . . 24
3.3 The Tenant Provisioner wizard . . . 24
3.3.1 Welcome page . . . 24
3.3.2 Tenant details . . . 25
3.3.3 Tenant administrator account . . . 25
3.3.4 Review and confirm . . . 25
3.3.5 Completion . . . 26
3.4 The Party Provisioner wizard . . . 26
3.4.1 Welcome page . . . 26
3.4.2 Root party . . . 26
3.4.3 Child parties . . . 27
3.4.4 Review and confirm . . . 27
3.4.5 Completion . . . 28
3.5 What comes next . . . 28

1. Introduction

1.1 About ORE Studio

ORE Studio is a desktop application for exploring, configuring, and running quantitative finance calculations. It provides a graphical environment built around the *Open Source Risk Engine* (ORE), a widely-used open-source library for pricing derivatives and measuring financial risk, which is itself built on QuantLib. ORE Studio handles the data — storing trades, market data, and model configurations, and presenting the results — while ORE and QuantLib provide the mathematics.

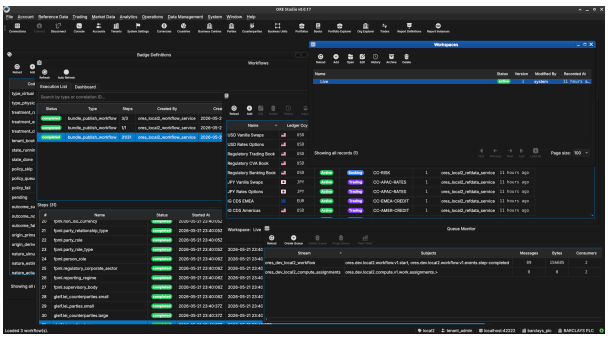


Figure 1.1: ORE Studio v0.0.17 — the main workspace showing the instrument and trade management views.

1.2 Audience

This manual is written for several kinds of reader:

- **Analysts, traders, quants, and middle-office staff** who want to explore financial instruments and risk calculations through a graphical interface. No programming experience is required.
- **Students and researchers** learning quantitative finance. ORE Studio makes it possible to experiment with derivative pricing, yield curve construction, and risk measures without writing code.
- **LLM-based agents and coding assistants** that interact with ORE Studio on a user's behalf, help configure workspaces, interpret results, or draft analytical reports. Multimodal

models — those capable of processing both text and images — are preferred, as the manual makes extensive use of screenshots to describe the user interface.

The focus throughout is on what you can do through the application itself: entering trades, configuring market data, running analytics, and understanding the results.

1.3 Functional Areas

ORE Studio is organised around a set of functional areas accessible from the main menu:

- *Reference data* — currencies, business calendars, counterparties, and market conventions that form the foundation for everything else.
- *Instruments and trades* — define, store, and manage financial instruments and their terms.
- *Market data* — yield curves, volatility surfaces, fixing histories, and other inputs to pricing and risk models.
- *Model configuration* — choose and parameterise pricing engines, simulation models, and sensitivity specifications.
- *Analytics* — run calculations and view results: net present value, sensitivities (Greeks), credit and funding valuation adjustments (XVA), Monte Carlo simulations, and stress tests.

All data is stored persistently, so trades, curves, and configurations can be saved, revisited, and reused across sessions.

1.4 What ORE Studio Is Not

ORE Studio is a learning and exploration environment, not a production trading or risk system. It is independent of and unaffiliated with ORE, QuantLib, or any financial institution. All quantitative mathematics are provided by ORE and QuantLib; ORE Studio is the surface through which they are configured and their results explored. Users who need production-grade performance, real-time market data feeds, or regulatory reporting should look to enterprise risk platforms.

1.5 How This Manual Is Organised

The chapters follow the natural order of first use. After this introduction, *Connecting to ORE Studio* covers launching the application, establishing a connection to the backend, and orienting yourself in the main window; *Initial Setup* then covers provisioning the system, tenants, and parties. Later chapters cover each functional area in turn. You do not need to read the manual cover to cover; each chapter can be read independently once the system is up and running.

1.6 Early-Stage Software Notice

Early-stage software. ORE Studio is currently in active development (version 0.x). Features, workflows, and this documentation are all evolving and may change between releases. Numbers produced by the system are for learning and exploration only — they must not be used for real trading, risk management, or any financial decision-making.

1.7 A Note on This Document

This manual was generated entirely by large language models (LLMs) working under human supervision. While every effort has been made to ensure accuracy, LLM-generated content can contain errors, omissions, or descriptions that do not match the actual software behaviour. If you find a discrepancy between this manual and the application, trust the application. Please report inaccuracies via the project issue tracker so they can be corrected.

2. Connecting to ORE Studio

2.1 Getting started

When you launch ORE Studio a splash screen appears briefly while the application initialises, showing the ORE Studio logo and the build version in its lower-right corner.

Figure 2.1: The ORE Studio splash screen shown during start-up. The build version appears in the lower-right corner.

Once initialisation completes you are greeted by the main window in its *disconnected* state. The application is running but has not yet established a connection to the backend service that performs calculations. Before you can work with trades, market data, or analytics, you need to connect to a running ORE Studio backend.

This chapter covers:

- What the main window looks like before a connection is established.
- How to open the Connection Manager and configure one or more backends.
- How to log in once a connection is active.
- How to manage your saved connections — adding, editing, deleting, organising by environment and tag.
- How to read the status bar and understand what each zone shows.
- How to launch ORE Studio from the command line with options for telling windows apart, configuration directory, auto-connect, and diagnostics.

2.2 The main window before login

In the disconnected state most of the application is inactive. The menu bar and toolbar are present but workspace panels, trade lists, and analytics views are all unavailable until a successful

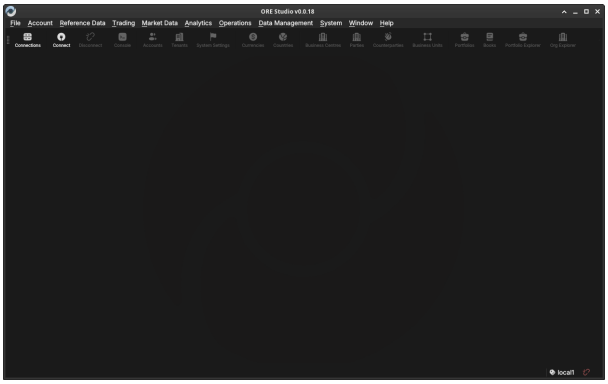


Figure 2.2: The ORE Studio main window immediately after launch, in the disconnected state. The toolbar shows the Connect and Connections buttons; the workspace area is empty until you log in.

login. Two toolbar buttons are always accessible:

- **Connect** — opens the login dialog for the currently selected connection.
- **Connections** — opens the Connection Manager, where you configure the list of backends ORE Studio knows about.

2.3 The Connection Manager

The Connection Manager is the central place where you maintain the list of ORE Studio backends the application can connect to. You might have a local development backend, a shared team backend, and a staging backend — each appears as a separate entry here. The manager lets you add, edit, delete, and organise those entries before you attempt a login.

Open it at any time from the toolbar **Connections** button or from the *File* menu.

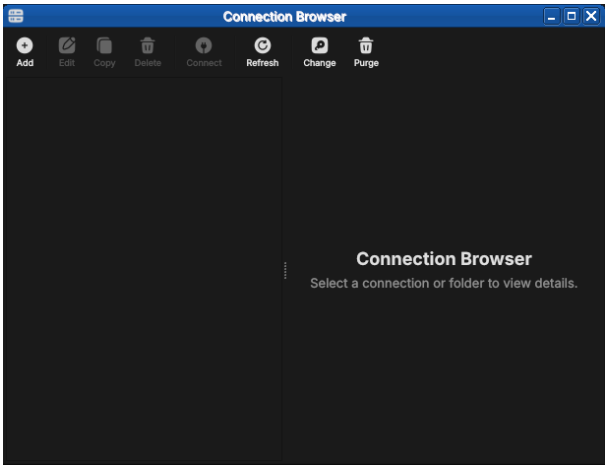


Figure 2.3: The Connection Browser open with no connections configured. The toolbar exposes Add, Edit, Copy, Delete, Connect, Refresh, Change and Purge; the unusable actions are greyed out until something is selected.

2.3.1 Adding a new connection

Click **Add** (or the "+" button) to open the *New Connection* form. Fill in the fields:

- **Name** — a free-text label you choose, used throughout the UI to identify this backend (e.g. "Local dev", "Team staging"). Must be unique among your saved connections.
- **Host** — the hostname or IP address of the machine running the ORE Studio backend (e.g. `localhost`, `192.168.1.10`, `ores-staging.example.com`).
- **Port** — the TCP port the backend is listening on. The default is `35900`; change it only if the backend was started on a different port.
- **Environment** — an optional grouping label (see Environments below). Helps you distinguish development, staging, and production backends at a glance.
- **Tags** — zero or more free-text labels (see Tags below). Useful for filtering and searching when you have many connections.

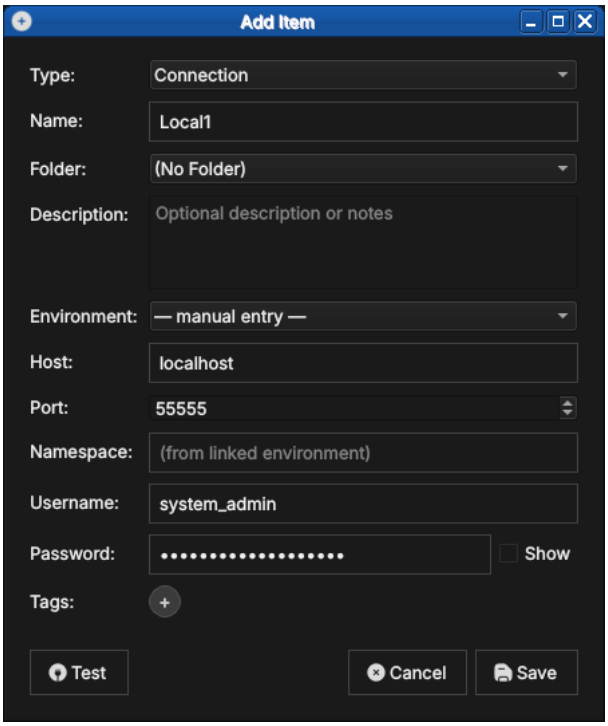


Figure 2.4: The New Connection form with example values filled in. *Type* is set to `Connection`; the *Environment* may be left as *manual entry* or linked to a defined environment.

Click **Save** (or **OK**) to add the connection to the list. The new entry appears immediately in the Connection Manager list. No network contact is made at this point — ORE Studio simply stores the configuration.

2.3.2 Editing a connection

Select an existing connection in the list and click **Edit** (or double-click the row) to open the same form pre-populated with the saved values. Change any field and click **Save**. The connection list updates immediately.

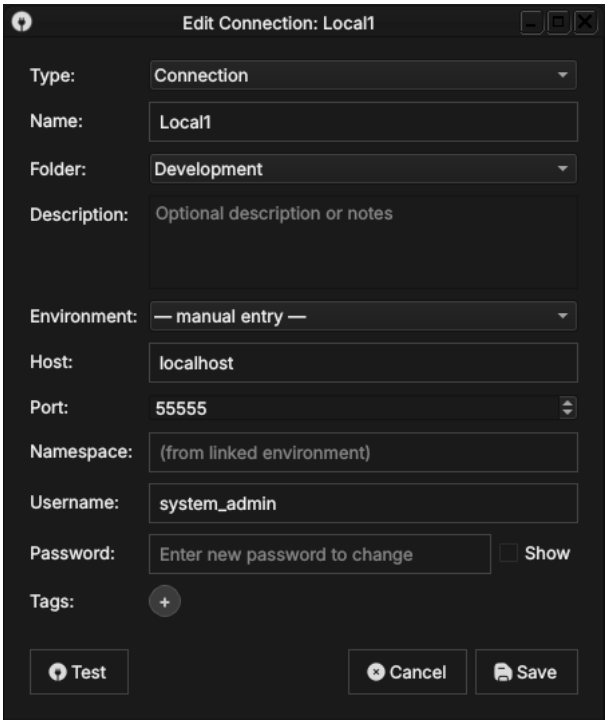


Figure 2.5: Editing a saved connection. The dialog title shows *Edit Connection*, the fields are pre-populated, and the password field reads *Enter new password to change* — the stored password is preserved unless you type a replacement.

2.3.3 Deleting a connection

Select one or more connections in the list and click **Delete** (or press the Delete key). A confirmation dialog asks you to confirm the removal.

Deleting a connection removes only the saved configuration; it has no effect on the running backend or any data stored there.

2.3.4 Organising connections into folders

When the list grows, *folders* keep it manageable. A folder is a named container you can nest, grouping related connections — by team, by client, or by environment tier. In the populated browser above, the connections sit under a `Development → Dev → Local1...Local4` folder tree.

To create one, click **Add** and set *Type* to `Folder`. Give it a name and an optional description, and choose a parent folder (or *No Folder* to place it at the top level). Connections are then assigned to a folder through their *Folder* field, or by dragging them in the tree.

2.3.5 Protecting your credentials with a master password

The passwords you save with a connection are encrypted at rest using a *master password* that you choose. The first time you save a credential, ORE Studio offers to create one.

Type the same value into *New Password* and *Confirm Password*; the fields turn green when they match. *Show password* reveals what you typed.

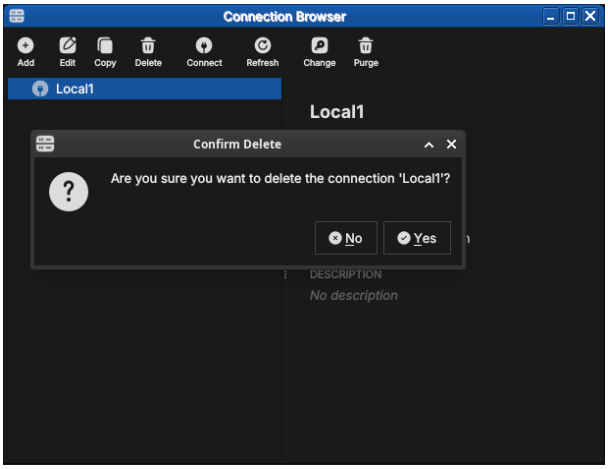


Figure 2.6: The delete confirmation dialog. ORE Studio names the connection being removed and waits for you to confirm with *Yes* or cancel with *No*.

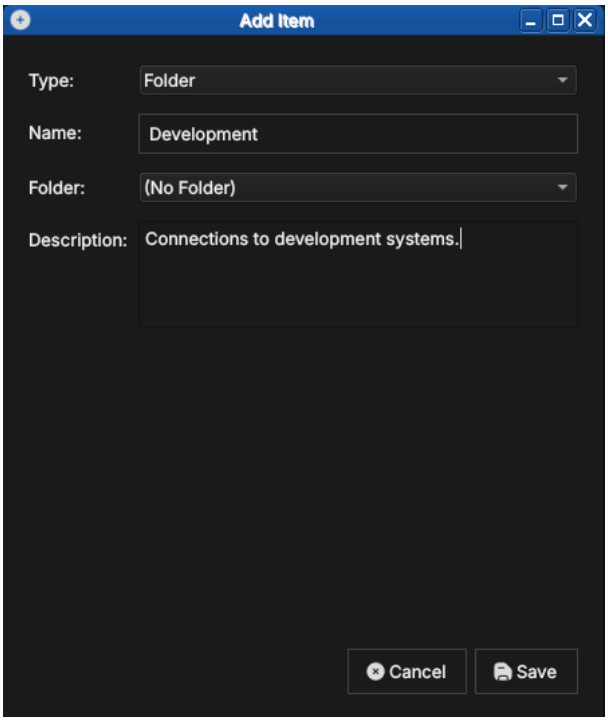


Figure 2.7: Creating a folder. With *Type* set to `Folder`, a folder needs only a name and an optional description; it can be nested inside another folder.

Figure 2.8: Creating the master password. It encrypts every saved server password; there is no recovery if you forget it, so you may also leave it blank to store passwords unencrypted.

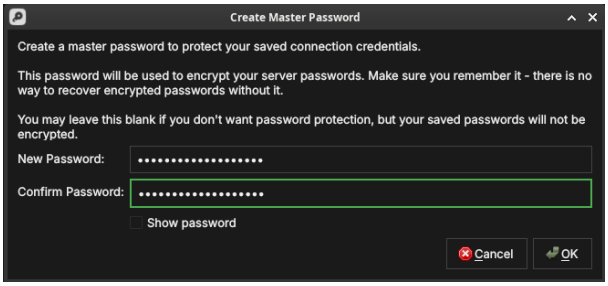


Figure 2.9: The master password confirmed — both fields match and are outlined in green, ready to accept with *OK*.

When you next launch ORE Studio and open the Connection Browser, it prompts you to unlock your saved connections by entering the master password. Until you do, the encrypted passwords stay sealed.

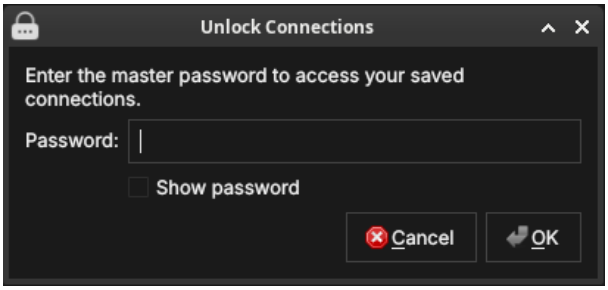


Figure 2.10: The unlock prompt shown on a later launch. Enter the master password to decrypt your saved connection credentials.

If you leave the master password blank, your connections still work but their passwords are stored unencrypted — acceptable on a personal machine, but not recommended on shared or portable systems.

2.3.6 Environments

An *environment* is a named grouping that you assign to a connection to indicate which tier of your infrastructure it belongs to. Typical values are `Development`, `Staging`, and `Production`, but the list is fully customisable — you can define any environment names that make sense for your setup.

Environments appear as a coloured badge or label next to the connection name in the list, making it immediately obvious which tier you are about to connect to. This is a safety feature: it is easy to accidentally connect to production when you meant staging; a clearly labelled environment badge prevents that.

To manage the available environment names, open the *Environments* section within the Connection Manager settings (or the dedicated menu item). From there you can:

- **Add** a new environment name and choose its display colour.
- **Rename** an existing environment (all connections using it update automatically).

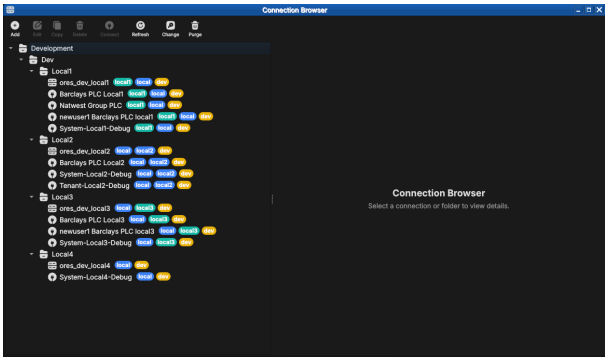


Figure 2.11: A populated Connection Browser. Connections are grouped under folders (here `Development → Dev → Local1...Local4`) and each carries coloured chips — the environment and tags — making the tier and purpose of every entry obvious at a glance.

- **Delete** an environment (connections that used it revert to *Unassigned*).

An environment is itself created from the Connection Browser: click **Add** and set *Type* to `Environment`. An environment carries its own host, port, HTTP port, namespace and tags — these become the connection details that any connection linked to it inherits.

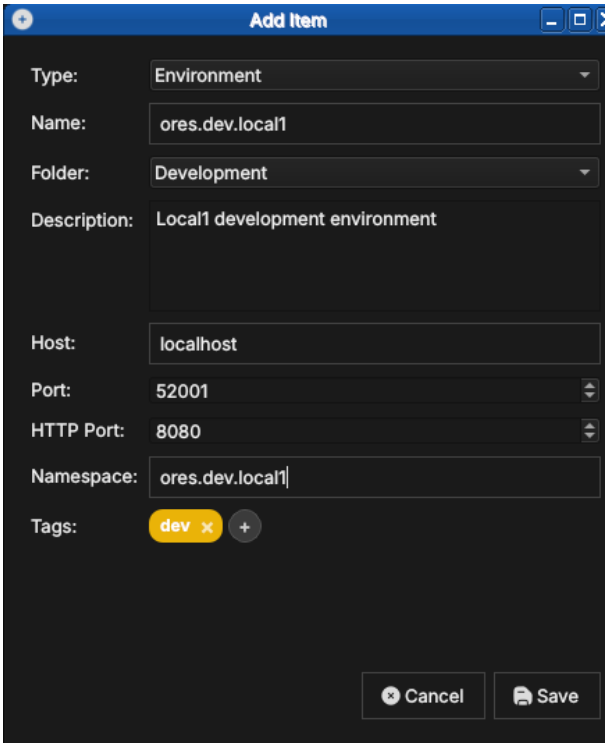


Figure 2.12: Creating an environment. With *Type* set to `Environment`, you give it a name, folder, host, ports, namespace, and tags (here a `dev` tag).

Once an environment exists, a connection can link to it by selecting it in the connection's *Environment* field instead of *manual entry*. The connection then inherits the environment's host, port, and namespace, so you maintain those details in one place.

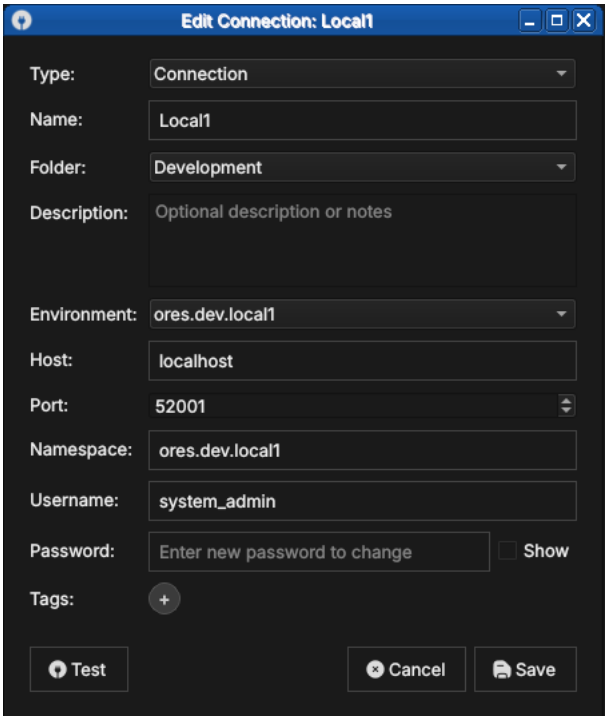


Figure 2.13: A connection linked to an environment. Selecting `ores.dev.local1` in the *Environment* field populates the host, port and namespace from that environment.

2.3.7 Tags

Tags are free-text labels you attach to a connection to enable flexible filtering and search. Unlike environments (which represent a single tier), a connection can carry any number of tags. Examples: `personal`, `client-acme`, `read-only`, `high-memory`.

Tags are displayed inline with the connection name in the list. The Connection Manager provides a tag filter bar at the top of the list: type or select a tag to show only the connections that carry it.

`../../../assets/images/connection_manager_tag_filter.png`

To add or remove tags on a connection, open the Edit form for that connection and use the tag input field. Type a new tag name and press Enter (or comma) to add it; click the × on an existing tag chip to remove it. Tags are created on first use — there is no separate tag management screen.

2.3.8 Where your connections are stored

ORE Studio keeps your connections — together with their environments, folders, and tags — in a single local SQLite database file named `connections.db`. Your UI settings (preferences and window layout) are stored separately, in the operating system's standard settings store. Neither

leaves your machine.

Operating system	Settings directory	Database location (`connections.db`)
Linux	`~/.config/OreStudio/`	`~/.local/share/ores.qt/connections.db`
macOS	`~/Library/Preferences/`	`~/Library/Application Support/ores.qt/connections.db`
Windows	Registry: `HKCU\Software\OreStudio\OreStudio`	`%APPDATA%\ores.qt\connections.db`

If `connections.db` is missing when ORE Studio starts, it creates a new empty one — so the Connection Manager simply opens with no entries.

2.3.9 Backing up and restoring your connections

Because everything you configure here lives in that one `connections.db` file, backing it up is a file copy:

1. Quit ORE Studio, so the database is fully written and not locked.
2. Copy `connections.db` from the location above to a safe place — on Linux, for example:

   ```
   cp ~/.local/share/ores.qt/connections.db ~/connections-backup.db
   ```

To restore, quit ORE Studio and copy the backup back over `connections.db`. To move your connections to another machine, copy the file to the same location there. The file is self-contained, so a single copy preserves all your connections, environments, folders, and tags.

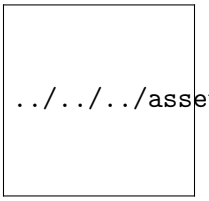
2.4 Connecting and logging in

Once you have at least one connection configured, you can log in.

2.4.1 Selecting the active connection

In the Connection Manager, select the connection you want to use and click **Set as active** (or double-click it). The selected connection becomes the target for the **Connect** button in the main toolbar. The status bar at the bottom of the main window shows the name of the active connection.

Alternatively, the **Connect** toolbar button opens a quick-select dropdown if no connection is active, letting you pick one without opening the full Connection Manager.



2.4.2 The login screen

With an active connection selected, click **Connect**. ORE Studio attempts to reach the backend. If the backend is reachable, the *Login* dialog appears.

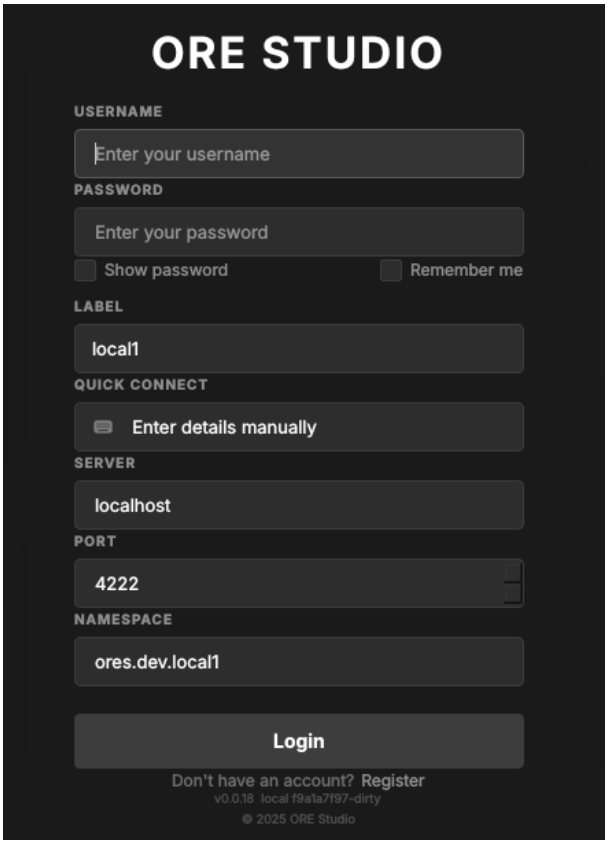


Figure 2.14: The login dialog. Below the credentials it shows the connection *Label*, a *Quick Connect* selector, and the *Server*, *Port* and *Namespace* the login will target; the footer carries the *Register* link and the build version.

Enter your **username** and **password** and click **Login**. Tick *Remember me* to have ORE Studio recall the username next time. ORE Studio authenticates against the backend's user database. On success the dialog closes and the main window transitions to the *connected* state: the workspace panels become active, the menu bar is fully enabled, and the status bar shows your username and the connected backend name.

The fields below the credentials describe *which* backend you are logging in to:

- **Label** — the saved connection's name (e.g. `local1`).
- **Quick Connect** — pick a saved connection to fill *Server*, *Port* and *Namespace* in one step, or leave it on *Enter details manually* to type them yourself.
- **Server**, **Port**, **Namespace** — the backend address the login targets.

If you do not yet have an account, the **Register** link in the footer opens account registration. If authentication fails, an error message is shown below the password field. Check that you are using the correct credentials for this backend; each backend maintains its own user database

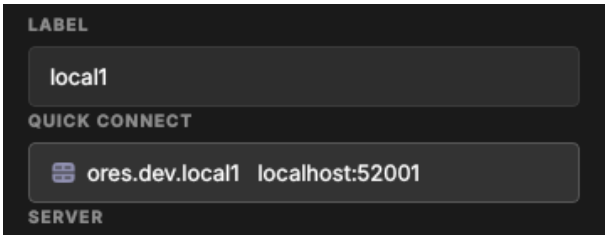


Figure 2.15: The *Quick Connect* selector. Choosing a saved entry fills in the server, port and namespace so you do not have to type them.

independently.

2.4.3 First login and the default account

On a freshly initialised backend the only account that exists is the default administrator account. Your system administrator will have provided the initial credentials. After first login it is strongly recommended to change the password immediately via *Settings* → *Account*.

2.5 Connection status and reconnection

Once connected, the status bar permanently shows the connection state. If the backend becomes unreachable (network interruption, service restart) ORE Studio displays a *disconnected* banner and disables workspace interactions. Use the **Connect** button to reconnect; your saved connection configuration is retained, so no re-entry of the host or port is needed — only your password.

../../../assets/images/connection_lost_banner.png

2.6 The status bar

The status bar runs along the bottom of the main window and provides a continuous read on the application's connection state. It is always visible regardless of what is open in the workspace.

../../../assets/images/status_bar_connected.png

The status bar is divided into zones from left to right:

- **Connection name** — the name of the active connection as entered in the Connection Manager (e.g. "Local dev"). Clicking this zone opens the Connection Manager directly.

- **Environment badge** — the coloured environment label (e.g. `Production` in red) if one is assigned to the active connection. Absent when no environment is set. This is the most prominent safety indicator: always glance at the environment badge before performing any write operation.
- **Username** — the account name of the currently logged-in user. Absent in the disconnected state.
- **Server version** — the version string reported by the backend service. Useful when reporting issues or verifying that the client and backend are compatible.
- **Session duration** — a running clock showing how long the current login session has been active.

The status bar changes appearance to reflect the connection state:

State	Appearance
Disconnected (no active connection)	Grey; shows "Not connected"
Connecting (handshake in progress)	Animated indicator; shows "Connecting to /name/..."
Connected and logged in	Normal; shows all zones as described above
Connection lost (backend unreachable)	Warning colour (amber/red); shows "Connection lost — *name*"
Reconnecting	Animated indicator; shows "Reconnecting..."

In the disconnected state the strip is reduced to the environment marker and the connection label, with the broken-link icon on the right:

Figure 2.16: The status bar in the disconnected state, showing the environment marker, the connection label (`local1`), and the broken-link indicator.

The fully connected state is shown at the start of this section; the connection-lost state is shown under Connection status and reconnection above. Compare the three to recognise at a glance which state the application is in.

2.7 Starting ORE Studio from the command line

ORE Studio can be launched directly from a terminal, which is useful for scripting, for telling several windows apart, or for pointing the application at a non-default configuration directory. Run `ores --help` to see the full list of accepted options for your installed version; the most commonly used ones are documented below.

```
ores --help
```

2.7.1 Telling windows apart (instance colour and name)

When you run more than one ORE Studio window at once — for example against different backends, or from separate checkouts — you can give each window a distinct identity so you can tell them apart at a glance. This is *not* a light/dark theme; it is purely a per-window marker.

- `--instance-color HEX` draws a small coloured circle in the status bar in that colour (a 6-digit RGB hex, e.g. `F44336` for red). Give each window a different colour.
- `--instance-name NAME` (short form `-n`) labels the window with a name, also shown in the status bar, so you can see which window is which.

```
ores --instance-name "Local dev" --instance-color 2196F3
```

The colour and the name are independent: the colour is only a visual marker, and the name is what identifies the instance. If you launch via `build/scripts/start-client.sh`, its `--colour red|green|blue|<hex>` sets the marker colour and `--name` sets the instance name; when you omit `--name`, the name comes from `ORES_CHECKOUT_LABEL` in your `.env` — never from the colour.

../../../assets/images/main_window_instance_colour.png

2.7.2 Configuration directory

By default ORE Studio stores its UI settings — preferences and window layout — in the platform's standard user configuration directory (`~/.config/OreStudio/` on Linux, `~/Library/Application Support/OreStudio/` on macOS, `%APPDATA%\OreStudio\` on Windows). Your saved connections live separately, in `connections.db` (see Where your connections are stored). To use a different directory:

```
ores --config-dir /path/to/config
```

This is useful when running multiple isolated instances, or when keeping configuration under version control for team-shared settings.

2.7.3 Selecting a connection at startup

To bypass the Connection Manager and connect immediately to a named connection:

```
ores --connection "Local dev"
```

ORE Studio will start, set the named connection as active, and open the login dialog automatically. The connection name must match exactly (case-sensitive) a saved entry in the Connection Manager.

2.7.4 Logging and diagnostics

For troubleshooting, verbosity can be increased:

```
ores --log-level debug    # verbose output to stdout
ores --log-file /tmp/ores.log  # write log to a file instead
```

Log output includes connection lifecycle events, authentication attempts, and backend protocol messages. The `debug` level is intended for issue reporting and development; it produces high-volume output and should not be left enabled during normal use.

2.8 Finding the application version

Knowing exactly which build you are running matters when reporting an issue or checking client/backend compatibility. ORE Studio shows its version in several places.

The title bar of the main window always shows the version next to the application name:

Figure 2.17: The version in the main window title bar, alongside the full menu bar and toolbar of the connected application.

It also appears in the lower-right corner of the splash screen at start-up:

Figure 2.18: The build version in the lower-right corner of the splash screen.

. . . and in the footer of the login dialog, below the *Register* link:

From the command line, `ores --version` prints the client version string (e.g. `ORE Studio 0.0.19`) and exits — handy in scripts or when filing a bug report.

```
ores --version
```

2.9 Running in the background and the system tray

While ORE Studio is running it places an icon in your desktop's system tray (notification area). The icon keeps the application reachable when its window is minimised or hidden, and shows the instance name so you can tell multiple windows apart.

2.10 Logging out

To end your session, choose *File* → *Log Out* or click the **Disconnect** toolbar button. ORE Studio closes the connection to the backend and returns to the disconnected main window. Your saved connection configurations are preserved; you can log back in at any time.

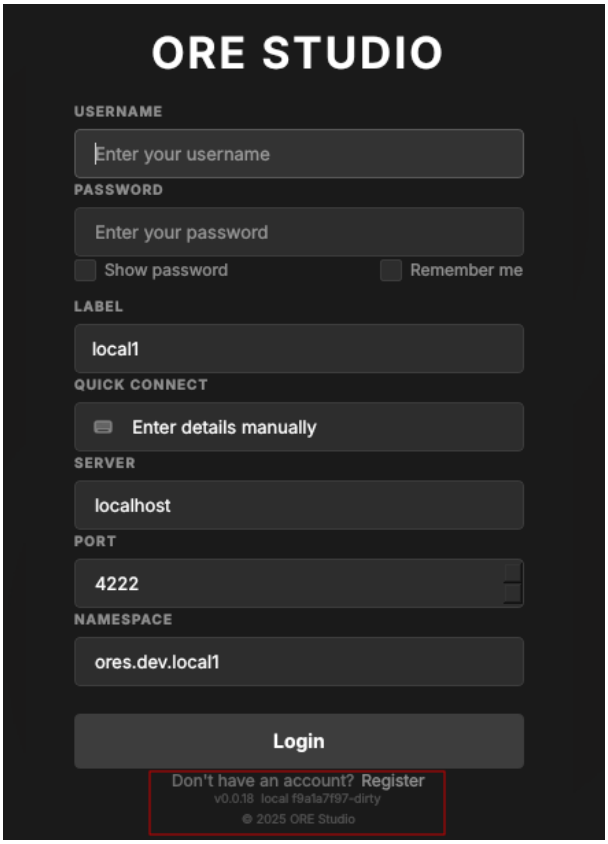


Figure 2.19: The version string in the login dialog footer, beneath the *Register* link.

Figure 2.20: The ORE Studio icon in the system tray, labelled with the instance name so you can identify the window it belongs to.

Logging out does not stop the backend service — it only terminates your client session. Other users connected to the same backend are unaffected.

3. Initial Setup

3.1 Understanding the multi-tenant, multi-party model

Before you can use ORE Studio for the first time, the system must be bootstrapped — meaning the initial administrative structure must be established. Understanding why this step exists requires a brief introduction to how ORE Studio organises data.

3.1.1 Tenants

A *tenant* is a fully isolated organisational space within an ORE Studio deployment. Each tenant has its own users, its own reference data (currencies, counterparties, books), and its own analytics results. Data belonging to one tenant is completely invisible to another tenant; there is no cross-tenant data leakage by design.

A typical deployment has one tenant per organisation. If you are running ORE Studio for your own firm, you will create one tenant representing your organisation. A managed-service provider running ORE Studio on behalf of multiple clients might create one tenant per client.

ORE Studio supports four tenant types, each with different operational controls:

Type	Purpose	Controls
System	Platform administration; one per deployment	Platform-managed
Production	Real customer organisations with live data	Strict
Evaluation	Demos, UAT, training, and exploratory testing	Relaxed
Automation	Programmatic test infrastructure; not for human use	None

The **system tenant** is special — it is created automatically during bootstrapping and cannot be deleted. It is the home of the platform administrator accounts. There is exactly one system tenant per deployment. All other tenants are created by platform administrators working from the system tenant.

Production tenants are for live operations. They enforce strict controls including four-eyes authorisation for sensitive operations and KYC-gated counterparty onboarding.

Evaluation tenants provide a realistic but relaxed environment for demonstrations, user acceptance testing, and training. Bulk data import from external sources (such as the GLEIF/LEI registry) is available in evaluation tenants but not in production tenants.

3.1.2 Parties

Within a tenant, a *party* is a business unit — a legal entity, a branch, a trading desk, or any other organisational subdivision. Parties form a hierarchy: a parent party can see all data belonging to its children and grandchildren; a child party sees only its own data and that of its own descendants.

Every tenant has exactly one **system party**, created automatically when the tenant is provisioned. The system party is the administrative home for tenant administrator accounts. It is not a business entity and should not be used for trading activity.

Business parties — the entities that actually own trades, counterparties, and analytics results — are **operational parties**. You create these after the tenant is provisioned, using the Party Provisioner.

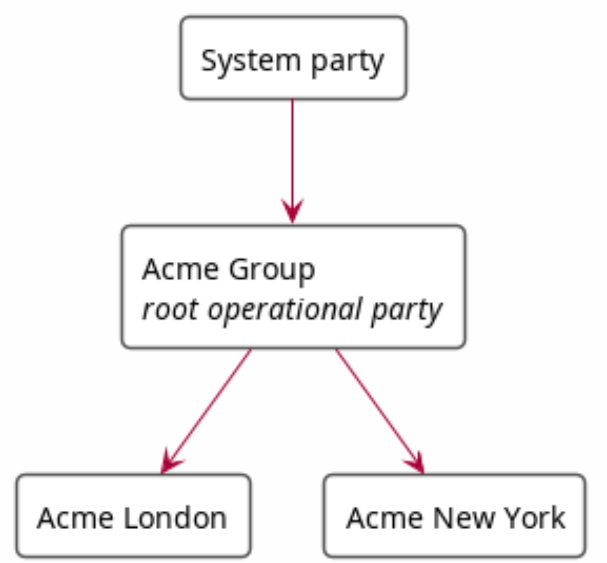


Figure 3.1: A simple party hierarchy: the system party at the top, the root operational party (Acme Group) below it, and two child operational parties (Acme London, Acme New York). Arrows point from parent to child.

A user logs in to a specific party within a tenant. Their data visibility is determined by their position in the hierarchy: a user logged in at "Acme Group" sees data from both "Acme London" and "Acme New York"; a user logged in at "Acme London" sees only their own data.

3.1.3 The bootstrapping sequence

A freshly installed ORE Studio backend goes through a fixed sequence before it is ready for normal use:

1. **System bootstrap** — the first platform administrator account is created and the system tenant is initialised. Until this step completes, the backend runs in *bootstrap mode* and accepts no normal user logins.
2. **Tenant provisioning** — one or more tenants are created by the platform administrator.

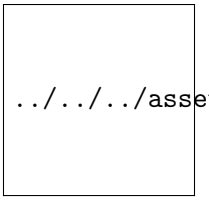
3. **Party provisioning** — within each tenant, the operational party hierarchy is established by the tenant administrator.

Each step has a corresponding wizard in the ORE Studio UI. The following sections walk through each one.

3.2 The System Provisioner wizard

The System Provisioner runs automatically when you connect to a backend that has never been initialised. It is a one-time wizard; once completed, it cannot be launched again from the same backend.

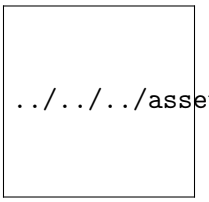
3.2.1 Welcome page



The welcome page confirms that the backend is in bootstrap mode and explains that you are about to create the first platform administrator account. Read it carefully — this account will have unrestricted access to the entire deployment.

Click **Next** to proceed.

3.2.2 Administrator account

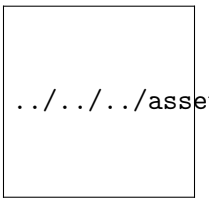


Fill in the fields:

- **Username** — the login name for the platform administrator (e.g. `admin`). This is case-sensitive. Choose something memorable; it cannot be changed after creation without database-level access.
- **Password** — a strong password for the administrator account. ORE Studio enforces a minimum length and complexity; the strength indicator shows the current rating as you type.
- **Confirm password** — re-enter the password to confirm. The Next button remains disabled until both fields match.

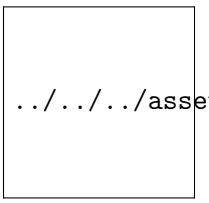
Click **Next**.

3.2.3 Review and confirm



The review page shows a summary of what will be created. Verify the username. Click **Back** to correct any mistake, or **Finish** to complete the bootstrap.

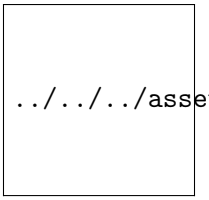
3.2.4 Completion



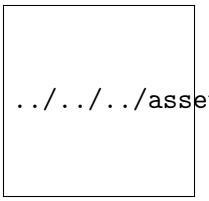
Once the wizard completes, the backend exits bootstrap mode. The System Provisioner closes and the Connection Manager or login dialog appears. Log in with the administrator credentials you just created.

3.3 The Tenant Provisioner wizard

After logging in as the platform administrator, you can create one or more tenants. Open the Tenant Provisioner from the **Administration** menu → *Provision Tenant*, or from the system dashboard if one is shown.

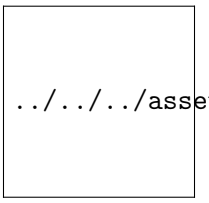


3.3.1 Welcome page



The welcome page explains that provisioning creates a new isolated tenant with its own system party and an initial administrator account for that tenant.

3.3.2 Tenant details

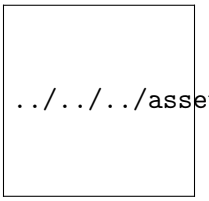


Fill in the fields:

- **Tenant name** — a human-readable name for the organisation this tenant represents (e.g. "Acme Group", "Client A"). This appears in the UI wherever the tenant is referenced.
- **Tenant type** — choose from *Production*, *Evaluation*, or *Automation*. See the table in Tenants above for the implications of each type. For a first-time setup representing a real organisation, choose *Production*. For a demo or test environment, choose *Evaluation*.
- **Description** — optional free-text notes about this tenant's purpose.

Click **Next**.

3.3.3 Tenant administrator account



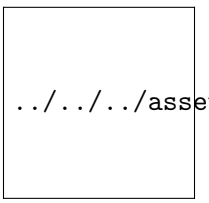
Create the initial administrator account for the new tenant:

- **Username** — the login name for this tenant's administrator. This is independent of the platform administrator account; each tenant maintains its own user database.
- **Password** / **Confirm password** — as with the system bootstrap, subject to the same strength requirements.

This account is associated with the new tenant's system party and is given the `TenantAdmin` role, which grants full control within the tenant but no cross-tenant access.

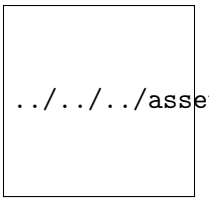
Click **Next**.

3.3.4 Review and confirm



The review page summarises the tenant and administrator that will be created. Click **Back** to correct any detail, or **Finish** to provision.

3.3.5 Completion



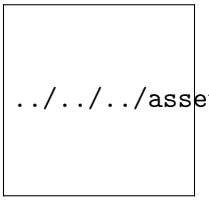
Provisioning creates the tenant record, the system party, and the administrator account. It also copies the standard set of system-wide permissions and roles into the new tenant so administrators can immediately assign roles to new users.

The new tenant now appears in the tenant list visible to the platform administrator. To begin setting up business parties, log out and log back in with the new tenant's administrator credentials, then run the Party Provisioner.

3.4 The Party Provisioner wizard

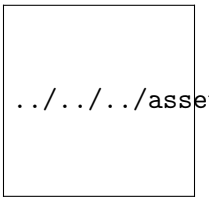
The Party Provisioner is run by the tenant administrator after initial tenant provisioning. It establishes the operational party hierarchy — the business units that will own trades, counterparties, and analytics. Open it from the **Administration** menu → *Provision Parties*, or from the tenant welcome screen on first login.

3.4.1 Welcome page



The welcome page explains that you are about to define the organisational structure your tenant will use. Unlike the system and tenant provisioners, the Party Provisioner can be run multiple times — you can always add more parties later via the main Reference Data → Parties screen.

3.4.2 Root party

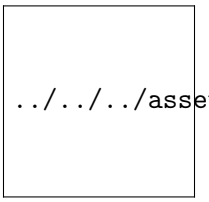


The root party is the top of your operational hierarchy — typically your holding company, group entity, or the single legal entity if your organisation has no subsidiaries.

- **Name** — the full legal or trading name of the root entity.
- **LEI** — the Legal Entity Identifier (a 20-character code issued by the GLEIF). Optional but strongly recommended for production tenants, where KYC workflows use the LEI to verify counterparty identity.
- **Description** — optional free-text notes.

Click **Next**.

3.4.3 Child parties



The child parties page lets you add operational parties beneath the root. This is optional — you can run the provisioner with just a root party and add children later through Reference Data → Parties.

To add a child party:

1. Select the parent in the tree (initially only the root is shown).
2. Click **Add child**.
3. Fill in the same fields as the root party (name, optional LEI, optional description).
4. Click **OK** to add it to the tree.

Repeat for as many parties as you need. Parties can be nested to any depth — branches under divisions under the holding company, for example.

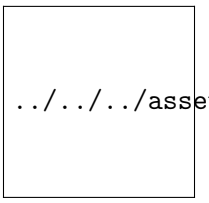
To remove a party from the tree before provisioning, select it and click **Remove**. Note: parties that have already been provisioned to the database can only be deleted through the main Parties screen, which enforces referential integrity checks.

Click **Next** when the hierarchy looks correct.

3.4.4 Review and confirm

The review page shows the full hierarchy as it will be created. Each party is shown with its name, level in the hierarchy, and LEI if provided. Click **Back** to modify the structure, or **Finish** to provision all parties in one operation.

3.4.5 Completion



Provisioning creates all parties in the correct parent-child order. The system party remains at the top of the hierarchy as the administrative root; all operational parties appear beneath it as its children or descendants.

Once provisioning is complete, users can be created and assigned to parties through the main Administration → Accounts screen. Trades, counterparties, and other business data are then entered or imported within the context of the party they belong to.

3.5 What comes next

With the system bootstrapped, at least one tenant provisioned, and the party hierarchy established, ORE Studio is ready for normal use. The next chapters cover:

- **Reference data** — currencies, calendars, and market conventions that must be set up before pricing can begin.
- **Instruments and trades** — defining and entering financial instruments.
- **Market data** — yield curves, volatility surfaces, and fixing histories.
- **Analytics** — running calculations and reviewing results.